



CoDAC: Algorithm-Hardware Co-Design by AutoML with Compression for Efficient Edge AI

Xingzhu Liu
School of Microelectronics
Shanghai University
Shanghai, China
lxz3984@shu.edu.cn

Zheng Lan
School of Microelectronics
Shanghai University
Shanghai, China
lz18373579282@shu.edu.cn

Guanhua Lu
School of Microelectronics
Shanghai University
Shanghai, China
guanhua_lu@yeah.net

Xiupeng Shi*
School of Microelectronics
Shanghai University
Shanghai, China
sxp@shu.edu.cn

Abstract

AI at the edge demands neural networks that are simultaneously accurate, resource-efficient, and readily deployable on hardware-constrained platforms. However, the disparate optimization of algorithmic models and hardware accelerators often leads to suboptimal system performance and high deployment costs. Herein, we present CoDAC, an automated algorithm-hardware co-design framework that unifies model pruning, quantization, and hardware-oriented optimization within a closed-loop AutoML pipeline. By leveraging fast, surrogate hardware feedback and multi-objective Bayesian optimization, CoDAC efficiently explores the co-design space, achieving optimal balances among accuracy, latency, and hardware resources, tailored for FPGA accelerators. Besides, the end-to-end toolchain automates the translation of optimized, compressed models into synthesizable hardware accelerators with minimal human intervention. Evaluated on the Xilinx Zynq-7020, CoDAC achieves substantial reductions in resource usage and inference latency while maintaining competitive accuracy, yielding Pareto-optimal solutions for diverse scenarios. CoDAC highlights the effectiveness and value of co-design AutoML in optimizing lightweight models and accelerator implementations from a systemic view.

CCS Concepts

• **Computing methodologies** → Artificial intelligence; Distributed artificial intelligence.

Keywords

Algorithm-Hardware Co-Design, Edge Intelligence, Neural Network Compression, AutoML, Chip Accelerator

ACM Reference Format:

Xingzhu Liu, Guanhua Lu, Zheng Lan, and Xiupeng Shi. 2025. CoDAC: Algorithm-Hardware Co-Design by AutoML with Compression for Efficient

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. AITC 2025, Hangzhou, China

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1862-5/2025/07

<https://doi.org/10.1145/3762329.3762343>

Edge AI. In *The 2nd International Conference on Artificial Intelligence of Things and Computing (AITC 2025)*, July 25–27, 2025, Hangzhou, China. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3762329.3762343>

1 Introduction

Artificial intelligence (AI) is increasingly deployed at the edge to enable applications ranging from smart roads and industrial intelligence, to AI of Things (AIoT)^{[1][2]}. These scenarios demand neural network (NN) models that offer not only high predictive accuracy but also ultra-low latency and stringent resource efficiency to meet the constraints of edge hardware^{[3][4]}. However, achieving optimal balances between algorithmic performance and hardware efficiency remains a central challenge for both academia and industry.

Recent advances in NN compression and hardware acceleration have substantially improved system efficiency^[5]. However, optimizing algorithms and hardware in isolation often falls short in real-world deployment. Most compression methods (e.g., pruning, quantization) typically prioritize software-centric metrics like parameter count and FLOPs, while often underestimating the specific properties of target hardware^[6]. Models that appear efficient in theory may perform inefficiently when implemented. Thus, a unified co-design method that integrates both algorithmic compression and hardware acceleration is needed, but remains lacking^[7].

Although hardware-aware co-design offers significant potential, the processes of hardware synthesis and profiling remain prohibitively time-consuming for large-scale exploration^[8]. Recent advances in automated machine learning (AutoML) have enabled optimized discovery of model architectures and compression strategies, but are generally hardware-agnostic^[9]. Moreover, even with well-compressed models, translating algorithms into efficient hardware implementations requires specialized expertise and considerable effort, posing a barrier to widespread edge AI advancement.

Herein, we introduce CoDAC, an AutoML-based framework for joint algorithm-hardware co-design, targeting lightweight NN compression and FPGA acceleration. The main contributions are as follows:

- Co-Design AutoML with Compression: A hardware-aware AutoML that jointly optimizes NN pruning, quantization,

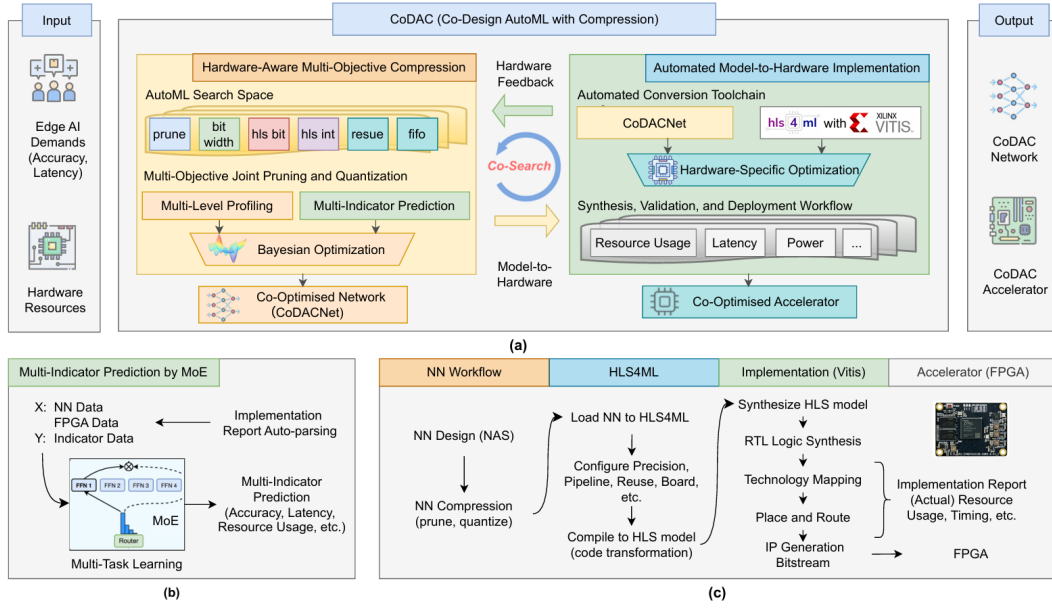


Figure 1: The Co-Design AutoML with Compression.

and hardware (FPGA) designs, resulting in co-optimized model-accelerator pairs.

- **Advanced Multi-Objective Co-Optimization:** We utilize multi-objective Bayesian optimization (BO) with multi-level profiling and surrogate hardware models to enable efficient and informed search.
- **Automated Accelerator Implementation:** We develop an automated toolchain that converts optimized models into synthesizable FPGA accelerators, with additional hardware-specific optimizations.

2 Methodology

2.1 CoDAC Framework

To enable co-optimized model-accelerator pairs for edge AI, we propose CoDAC, an automated algorithm-hardware co-design framework that unifies model pruning, quantization, and hardware-oriented optimization within a closed-loop AutoML pipeline, as illustrated in Figure 1. CoDAC is tailored to optimize and deploy lightweight NN models for resource-constrained edge AI systems, specifically targeting FPGAs and related ASIC hardware.

CoDAC co-tunes NN models and hardware accelerators to maximize overall system performance. At each iteration, multi-objective BO explores candidate configurations across pruning, quantization, and hardware settings. Multi-level profiling identifies critical factor relationships, refining the search space. A Mixture-of-Experts (MoE) model rapidly predicts key co-design indicators, including accuracy (A), latency (L), and resource utilization (H), significantly accelerating the search process. Additionally, an automated toolchain then converts optimized models into synthesizable FPGA accelerators, with further hardware-specific optimizations.

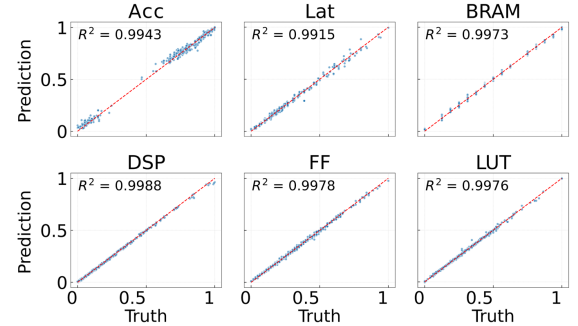


Figure 2: Co-design indicator multi-task prediction by MoE.

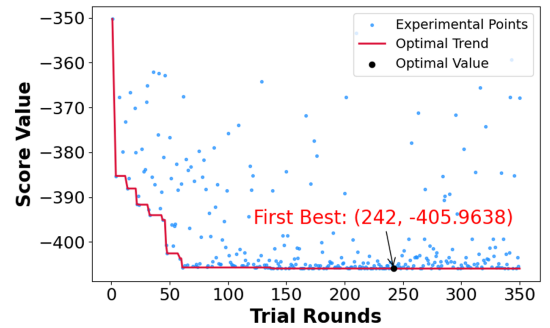
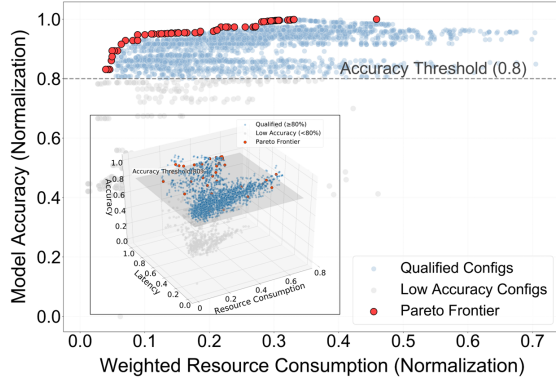


Figure 3: MOO Process by AutoML.

Table 1: Key Hyperparameters in Co-Design AutoML.

Hyperparameter	Symbol	Distribution	Value Range	Note
<i>Algorithm</i>				
Prune Sparsity	sparsity	Continuous uniform	[0.0-0.8]	Control parameter redundancy and sparsity.
Quantized Bit Width	bit_width	Discrete uniform	[3-16]	Specified for each model or layer.
...				
<i>Hardware (FPGA)</i>				
HLS Synthesis Total Bits	hls_bit	Discrete uniform	[3-16]	Affect logic resource usage.
HLS Integer Bits	hls_int	Discrete uniform	[0.25, 0.75]	Should match the quantization bit width.
Parallel Reuse Factor	reuse	Discrete uniform	[1, 100]	Balance computational parallelism and resource.
FIFO Depths	fifo	Discrete uniform	[1, 10]	Affect latency. Follow a power-of-two distribution.

**Figure 4: Pareto frontier results.**

2.2 Hardware-Aware Multi-Objective Compression

2.2.1 Multi-Objective Joint Pruning and Quantization. CoDAC adds hardware feedback into the compression process, enabling adaptive pruning and quantization to enhance both efficiency and hardware compatibility. To optimize the decisions, the AutoML systematically identifies optimal co-design hyperparameters within a bounded search space, balancing multiple objectives, as follows:

$$\begin{aligned}
 \Theta^* &= \operatorname{argmin}_{\Theta} \sum MOO_i \left(f(x|\alpha, C_p^{(\alpha)}, C_q^{(\alpha)}, h) \right) \\
 \Theta &\in \operatorname{SearchSpace}(\alpha, p, q, h) \\
 MOO &= \{A, H, L\} \\
 H &= \operatorname{ResourceUsage}(LUTs, DSPs, BRAMs, FFs, \dots) \\
 C_q^{(\alpha)} &= \operatorname{Quantize}(\alpha; q) \\
 C_p^{(\alpha)} &= \operatorname{Quantize}(\alpha; p)
 \end{aligned} \tag{1}$$

where α, p, q, h represents tunable hyperparameters in terms of network architecture, pruning, quantization, and hardware settings, as listed in Table 1. Key Hyperparameters in Co-Design AutoML.. CoDAC supports joint exploration of unstructured (e.g., individual weight removal) and structured (e.g., block-, group-wise) pruning ($C_p^{(\alpha)}$), and mixed-precision quantization at fine granularity ($C_q^{(\alpha)}$).

To balance multiple, potentially conflicting criteria, *MOO* serves as the aggregation method for multiple objectives, evaluated using the following metrics:

- **Accuracy (Acc):** Prediction accuracy of the compressed model, representing algorithmic performance.
- **Resource Efficiency:** FPGA main resource metrics, e.g., usage of Look-Up Tables (LUTs), Digital Signal Processors (DSPs), flip-flops (FF), and Block RAM (BRAM).
- **Latency (Lat):** End-to-end inference time, measured from memory input to final output.
- **Pareto-Optimal Frontiers:** Represent the best trade-offs among competing objectives.

With fast hardware feedback and automated tuning, CoDAC facilitates efficient, hardware-friendly compression schemes that maximize overall system performance.

2.2.2 Hardware-Friendly Compression Strategy. CoDAC integrates direct hardware feedback into both pruning and quantization processes to create models that are not only compact and accurate but also tailored for optimal hardware performance.

During pruning, CoDAC aligns sparsity patterns to match the parallelism and memory access properties of the target hardware, thereby maximizing computational throughput and resource efficiency. For quantization, a hardware-aware mixed-precision approach is adopted, allocating bit-widths to model components according to their quantization sensitivity and hardware feedback. Thus, compression is applied where it most benefits performance.

The hardware-friendly compression strategy is formulated as follows:

$$\begin{aligned}
 A' &= A_{\text{target}} - \alpha \cdot (1 - \rho) - \beta \cdot \left(\frac{1}{2^{b_q}} \right) \\
 H' &= \rho \cdot \frac{b_q}{b} \cdot H_{\text{original}} \\
 L' &= \frac{1}{k} \cdot \rho \cdot L_{\text{original}}
 \end{aligned} \tag{2}$$

where α and β are sensitivity coefficients calibrated experimentally; k is the acceleration factor, mainly determined by hardware capabilities; b_q denotes the quantization bit width, and ρ represents the pruning retention rate.

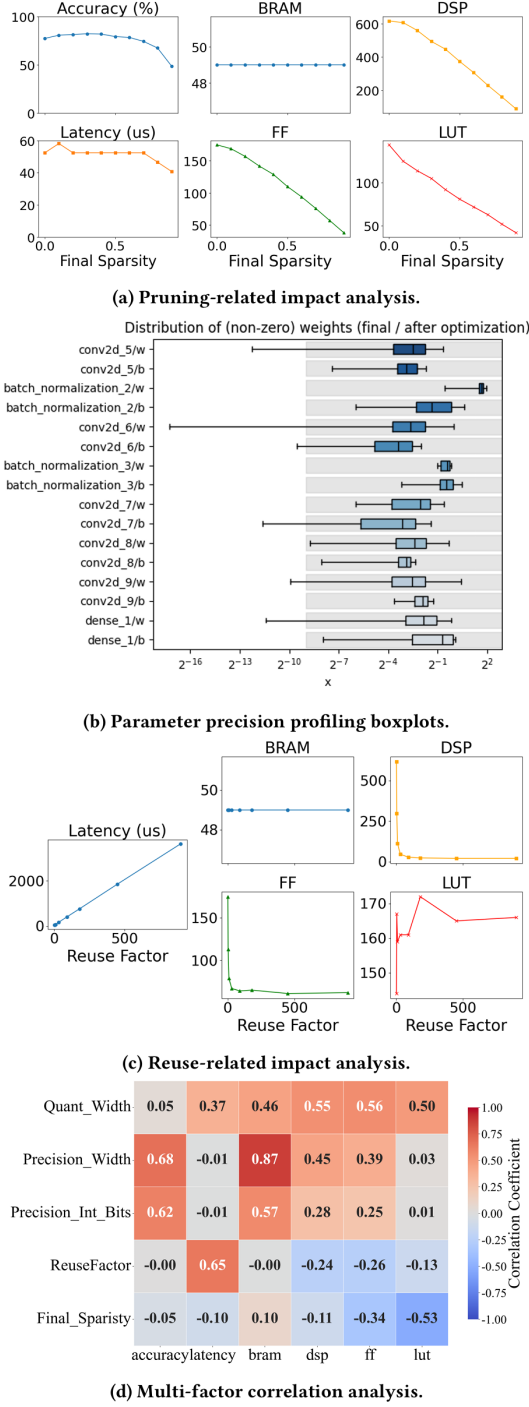


Figure 5: Co-design hyperparameter impact analysis.

Unlike conventional approaches that optimize for surrogate metrics such as model size or FLOPs, CoDAC ensures that model optimization aligns with actual system-level efficiency.

2.2.3 Multi-Level Profiling. We use multi-level profiling (e.g., sensitivity, corrective analysis) to systematically evaluate the impact of co-design factors (x) on system performance (y), as follows:

$$s = \frac{\partial x}{\partial y} \times \frac{x}{y} \quad (3)$$

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot (y_i - \bar{y})^2}}$$

Profiling is performed at multiple granularities, such as module (parameters, layers, blocks) and system levels, enabling a comprehensive understanding of relationships among algorithmic metrics, parameter counts, hardware resource usage, and reuse patterns.

Insights gained from profiling directly refine the AutoML design. Specifically, empirical statistics (e.g., the distributions of activations and weights before and after compression) are leveraged to guide the construction of the search space (e.g., ranges of bit widths).

2.3 Co-Design AutoML with Compression

2.3.1 AutoML Procedure. We address the co-design with compression by AutoML with multi-objective optimization (MOO), as depicted in Figure 1(b). The procedure integrates several key components: (1) pruning and quantization-aware training (QAT) to reduce model size while maintaining accuracy; (2) a mixture-of-experts (MoE) surrogate model for efficient and accurate prediction of co-design indicators, particularly hardware metrics; and (3) joint design space exploration using BO for optimal solutions.

2.3.2 Multi-Objective BO. The MOO method integrates BO, hybrid Pareto front update, and MoE-based feedback. Guided by Pareto-optimality, this step identifies the configurations yielding the best trade-offs among competing objectives, as follows:

$$\begin{aligned} & \text{Minimize } F(x) = [f_1(x), f_2(x), f_3(x)]^T \\ & \text{Subject to } C_j(x) \leq H_j, \quad j = 1, \dots, h \\ & P = \{x \in X \mid \text{no } x' \in X \text{ dominates } x\} \end{aligned} \quad (4)$$

where $C_j(x)$ and H_j are related to the hardware constraints. A solution $x \in X$ is Pareto-optimal if no other solution $x' \in X$ exists such that: $f_i(x') \leq f_i(x) \forall i$, and $f_j(x') < f_j(x)$ for at least one j .

The hybrid Pareto front update is designed to rapidly identify promising candidates near the Pareto frontier and enhance algorithmic performance through selective retraining. The retraining is to enhance reliability, while the remaining rely on predicted values from MoE for greater efficiency. The process is as follows:

$$F_{\text{new}}(x) = \begin{cases} f_m^{\text{Retraining}}(x), & \text{if } f_i(x)^{(t)} - f_i(x)_{\text{bst}}^{(t-1)} > \Delta \\ f_m^{\text{MoE}}(x), & \text{otherwise} \end{cases} \quad (5)$$

2.3.3 Co-Design Multi-Indicator Prediction. A data-driven surrogate model is trained on synthesized model-hardware pairs to rapidly predict performance indicators for unseen configurations, thus circumventing the prohibitive cost of full hardware synthesis for each candidate.

The workflow proceeds as follows: profiling to identify key hyperparameters $\Rightarrow$ FPGA synthesis $\Rightarrow$ automated extraction of indicator data by auto-parsing $\Rightarrow$ MoE model training $\Rightarrow$ validation, update, or retraining.

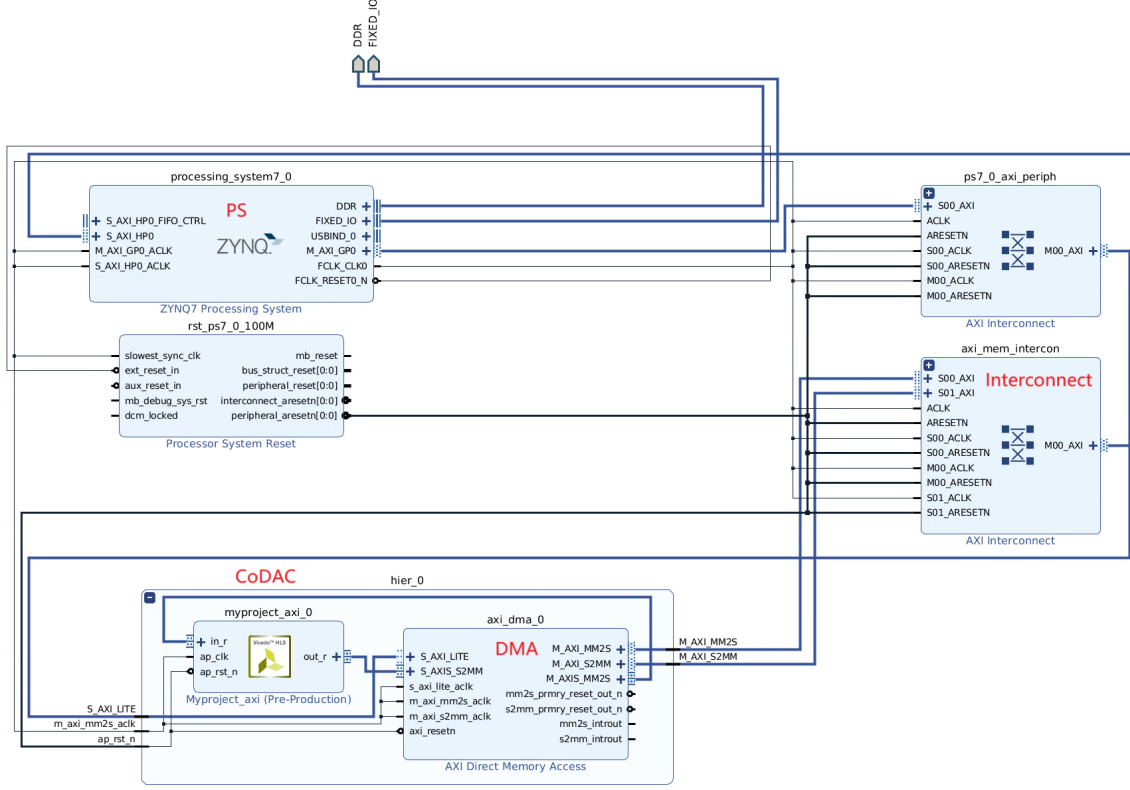


Figure 6: Hardware system architecture with CoDAC accelerator block.

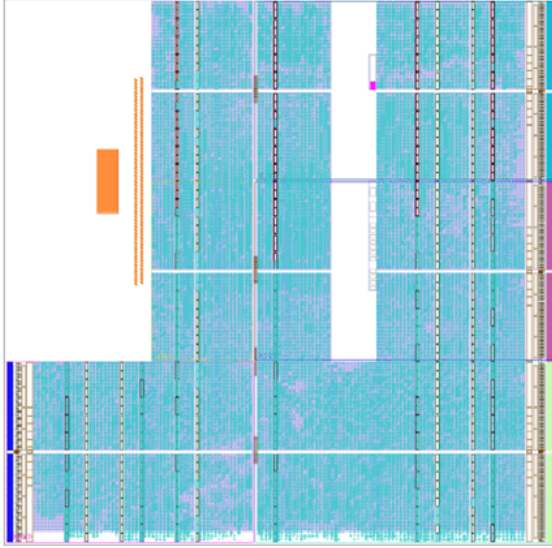


Figure 7: Physical design layout and routing diagram.

A small yet representative set of co-design factors (x_a, x_h) is selected and fully synthesized to collect ground-truth labels $y_m^{\text{synthesis}}$,

for indicator m . To automate data extraction from FPGA synthesis reports (rpt), we developed a parsing program $f_m^{\text{parsing}}(\text{rpt}) \rightarrow y_m^{\text{synthesis}}$. MoE then learns and fast approximates via multi-task functions, as follows:

$$\begin{aligned}
 f_m^{\text{MoE}}(\cdot) &= \text{MoE}(X_a, X_h, Y_m^{\text{synthesis}}) \\
 \widehat{y}_m &= f_m^{\text{MoE}}(x_a, x_h) + \epsilon \\
 \widehat{y}_m^{(k)} &= \sum_{i=1}^M g_i(x) \cdot f_i(x) \\
 \text{Minimize } \mathcal{L}_{\text{MoE}} &= 1 - R^2(Y_m^{\text{synthesis}} - \widehat{Y}_m)
 \end{aligned} \tag{6}$$

where $\widehat{y}_m$ denotes the predicted value, including algorithmic performance, resource usage, and latency. M is the number of experts, $f_i(x)$ represents the output of i -th expert for task m , and $g_i(x)$ is the gating function (softmax), indicating the contribution of expert i .

Since co-design indicators are interdependent, from a model training perspective, they share certain foundational features, but conflicts may arise across tasks, thus requiring specialized learning for each indicator. The MoE-based model is specifically designed to handle multi-task prediction by assigning subsets of its parameters (experts) to different tasks.

2.4 Automated Model-to-Hardware Implementation

2.4.1 Automated Conversion Toolchain. We develop an automated toolchain to optimize the conversion of compressed NN models into hardware-ready designs, enabling rapid prototyping and deployment, as depicted in Figure 1(c). By integrating HLS4ML with Xilinx Vitis, the toolchain translates quantized and pruned models into high-level synthesis (HLS) code and synthesizable register-transfer level (RTL) representations, streamlining the model-to-hardware process.

The hardware-related settings (e.g., bit width, computational reuse) are configured to address diverse application needs:

- **High-Parallelism:** Lower reuse factors to increase concurrent execution and minimize latency.
- **Resource-Efficiency:** Increase unit sharing to reduce FPGA resource usage for limited hardware.
- **Precision Tuning:** Adjust numerical representation to balance model accuracy and hardware cost.

2.4.2 Hardware-Level Optimization. While algorithmic compression reduces the computational and memory footprint of NN, further gains require hardware-level optimization. A notable inefficiency arises from oversized FIFO (First-In, First-Out) buffers used for data streaming between layers. Standard design practices often allocate excessive buffer depths, leading to overconsumption of on-chip resources such as BRAM and LUTs, particularly in deep or irregular architectures.

Thus, we introduce an automated FIFO depth optimization step in CoDAC. Using RTL co-simulation with initially large FIFO depths, we empirically monitor the peak occupancy of each buffer under representative workloads. Each FIFO is then resized to its minimal safe depth, defined as the maximum observed occupancy plus a safety margin (e.g., value = 1). This method aligns buffer sizes with actual dataflow demands, ensuring correctness while minimizing memory usage.

2.4.3 Synthesis, Validation, and Deployment Workflow. After architectural configuration and hardware-level optimization, the toolchain synthesizes the design into RTL code and generates FPGA-compatible IP cores using Vitis or Vivado. The resulting hardware is validated through emulation to verify functional correctness and expected performance.

CoDAC greatly accelerates the transition from algorithm optimization to hardware deployment, eliminating manual intervention. The workflow supports a range of lightweight NN architectures and multiple FPGA platforms, enabling flexible and fast deployment for diverse edge AI scenarios.

3 Analysis

3.1 Data and Experiment Settings

In our experiments, we target the Xilinx Zynq-7020 FPGA, a widely used platform for low-power AI inference in resource-constrained scenarios. We implement a lightweight network based on the ResNet architecture and evaluate performance on the SVHN dataset, which is representative of simplified classification tasks commonly encountered at the edge. This setting closely represents real-world

scenarios where edge devices handle moderate workloads, often as part of edge–cloud collaborative systems.

3.2 Co-Design by AutoML

3.2.1 Profiling-Informed Search Space Configuration. Multi-level profiling informs the search space configuration, as detailed in Section 3.3. The resulting search space is as defined in Table 1.

3.2.2 Indicator Multi-Task Prediction. For effective co-design AutoML, fast and precise hardware feedback is essential. This is achieved through the MoE-based multi-indicator prediction. As shown in Figure 2, the MoE model delivers robust and balanced performance across all six tasks, outperforming typical alternatives. Besides, the MoE model reduces hardware feedback time to <1 millisecond, an order-of-magnitude speedup over the HLS synthesis process, which typically takes >400 seconds per iteration.

The MoE model dynamically assigns specialized experts to individual indicator prediction tasks, effectively handling variations in task complexity and data distribution. This targeted expert routing enables the model to learn task-specific patterns efficiently, minimizing cross-task interference and enhancing overall predictive accuracy.

3.2.3 MOO Analysis. The co-design AutoML process leverages TPE for efficient search. Key objectives are aggregated using a weighted sum of normalized indicators, ensuring that incremental improvements in each metric are proportionally represented. Herein, the aggregated objective can be formulated as: $(1 - y^{Acc}) + (0.4 \cdot y^{LUT} + 0.3 \cdot y^{DSP} + 0.2 \cdot y^{FF} + 0.1 \cdot y^{BRAM}) + y^{Lat}$. The weights are set based on the relative importance of each metric in the overall optimization.

As depicted in Figure 3 and Table 2, the optimization process converges rapidly, and the resulting Pareto-optimal frontier demonstrates diverse trade-offs between accuracy and hardware efficiency, highlighting the effectiveness of the co-design AutoML.

3.2.4 Co-Design Performance. The Pareto front yields a diverse set of solutions, ranging from ultra-lightweight models for stringent resource constraints to high-accuracy variants suited for relaxed hardware requirements.

Under stringent resource constraints, co-tuning for accuracy and parallelism achieved substantial savings. At 11-bit quantization with a reuse factor of 27, DSP usage dropped from 655 to 51 units while preserving 78.4% accuracy, matching the full-precision baseline.

For latency-critical applications, 4-bit quantization and maximal parallelism (reuse factor = 1) lowered LUT usage to 26,068 units (an 66.8% reduction) and achieved an inference latency of 40.76 μ s. Accuracy decreased moderately from 81.5% to 75.8% (a 5.7% drop), still within acceptable bounds.

These results and Figure 4 demonstrate the effectiveness of co-design AutoML in jointly optimizing multiple factors to address diverse system needs.

3.3 Hyperparameter Impact Analysis

3.3.1 Pruning-Related Trade-offs. Pruning removes redundant NN connections, reducing hardware resource requirements. As shown in Figure 5(a), as sparsity increases, computational demands decline

Table 2: Results of MOO Pareto frontiers.

bit_width	hls_bit	hls_int	reuse	sparsity	accuracy	latency	bram	dsp	ff	lut
4	11	5	13	0.3726	78.52	55.98	26	8	22	47
4	11	5	15	0.3055	77.67	55.78	26	8	23	48
4	12	6	5	0.2241	76.37	48.63	26	7	23	49
6	13	8	27	0.3842	78.60	74.44	30	12	29	57

while accuracy is maintained up to a sparsity level of 0.6; further pruning results in a sharp decline in accuracy.

Pruning lowers the usage of arithmetic units (DSPs), storage registers (FFs), and logic gates (LUTs), but on-chip memory (BRAM) remains relatively stable. Latency follows a non-monotonic trend, with a slight initial increase due to sparsity-induced overhead, followed by a gradual decrease at higher sparsity levels.

These findings inform strategies for selecting the optimal sparsity point, with a focus on resources most sensitive to pruning (e.g., DSPs, FFs, LUTs).

3.3.2 Quantization-Driven Tuning. Quantization critically impacts hardware resource utilization, especially BRAM consumption. Both total bit-width and integer bit allocation exhibit strong positive correlations with BRAM usage (coefficients = 0.87 and 0.57, respectively) and moderate correlations with DSP, FF, and LUT resources, as shown in Figure 5(d). For resource-limited applications, appropriately reducing bit precision is beneficial.

Besides, parameter profiling refines the quantization bit-width search space, ensuring coverage of critical values while reducing search complexity and preventing overflow or truncation errors. As illustrated in Figure 5(b), most activations and weights fall within the representable range of the chosen integer and fractional bit-widths (indicated by the grey box).

Although per-layer or per-module quantization is feasible, we defer fine-grained tuning to future work due to BRAM’s linear sensitivity to system-wide quantization.

3.3.3 Resource-Reuse Sensitivity. Latency scales linearly with the reuse factor, revealing a key trade-off: greater reuse conserves DSP, FF, and LUT resources but slows computation due to increased sequential processing and dependencies, as shown in Figure 5(c). Optimal tuning seeks to minimize arithmetic and logic resource usage without incurring substantial latency penalties.

As shown in Figure 5(d), we observe a strong positive correlation between the reuse factor and latency (coefficient = 0.65), and a slight negative correlation with resource consumption. Notably, BRAM usage exhibits low sensitivity to reuse adjustments. This differential sensitivity provides insights for resource-specific optimization strategies.

3.3.4 System-Wide Optimization. Optimal performance requires the simultaneous co-optimization of multiple design factors. In resource-constrained settings, lower-precision representations and pruning reduce BRAM, LUT, DSP, and FF demands, while higher reuse factors further decrease hardware usage for applications that can tolerate increased latency.

As reflected in Table 2, this multi-factor optimization reduces resource utilization by 70% and latency by 12%, with less than

1.01% accuracy degradation, substantially outperforming single-technique baselines. Ablation studies confirm these synergistic benefits. Additionally, our AutoML-based optimization demonstrates clear efficiency gains.

3.4 Deployment Validation and System Insights

The optimized models are automatically converted into FPGA accelerators by the developed toolchain, with additional hardware-specific optimizations.

3.4.1 Hardware-Specific Optimization. We tune FIFO depths based on the dataflow. By analyzing the specific communication and throughput requirements between layers, we adjust FIFO sizes to closely match actual dataflow patterns, reducing redundant memory allocation. This strategy decreased BRAM usage from 46 units to 11. This improvement allows for either larger models or greater batch sizes within the same hardware constraints, making edge deployment more practical and cost-effective.

3.4.2 Accelerator Integration and Validation. After co-design, the NN is implemented as a custom accelerator IP core. HLS generates the RTL code, which then undergoes logic synthesis, placement, routing, and power analysis. This workflow produces a heterogeneous FPGA architecture, integrating specialized compute units, hierarchical memory, and optimized data paths, as shown in Figure 6 and Figure 7.

The Processing System (PS) manages accelerator configuration and runtime operations through dedicated control interfaces. NN computations are executed by a custom IP core, tightly integrated with a high-performance Direct Memory Access (DMA) controller. Data transfer between external DDR memory and the accelerator relies on the AXI Stream protocol, enabling high-bandwidth, low-latency communication and supporting efficient system scalability.

Empirical evaluation demonstrates that the deployed models are functionally correct, reliable, and capable of delivering the required performance within hardware constraints.

4 Conclusion

This paper presents CoDAC, an automated framework that unifies neural network compression with hardware-aware co-design for efficient edge AI systems. By integrating pruning, quantization, and FPGA-specific optimization into a closed-loop AutoML pipeline, CoDAC enables fast, multi-objective exploration that directly aligns algorithmic performance with hardware efficiency. Experimental results demonstrate that CoDAC delivers significant reductions in resource usage and latency with minimal accuracy loss, achieving Pareto-optimal performance for diverse scenarios.

The key value of CoDAC lies in its holistic, automated, and hardware-informed co-design methodology, which makes edge AI both efficient and practical. This not only accelerates the development and deployment of high-performance, resource-efficient AI models but also lowers the barrier to real-world edge intelligence. Future work will extend CoDAC to broader AI tasks and hardware platforms, further advancing scalable and accessible edge AI solutions.

Acknowledgments

This study is supported by the Shanghai Key Laboratory of Chips and Systems for Intelligent Connected Vehicles.

References

- [1] Xiupeng Shi, Yiik Diew Wong, Chen Chai, Michael Zhi-Feng Li, Tianyi Chen, and Zeng Zeng. Automatic clustering for unsupervised risk diagnosis of vehicle driving for smart road. *IEEE Transactions on Intelligent Transportation Systems*, 23(10):17451–17465, 2022.
- [2] Shuiguang Deng, Hailiang Zhao, Weijia Fang, Jianwei Yin, Schahram Dustdar, and Albert Y Zomaya. Edge intelligence: The confluence of edge computing and artificial intelligence. *IEEE Internet of Things Journal*, 7(8):7457–7469, 2020.
- [3] Claudionor N Coelho, Aki Kuusela, Shan Li, Hao Zhuang, Jennifer Ngadiuba, Thea Klaeboe Aarrestad, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers. Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors. *Nature Machine Intelligence*, 3(8):675–686, 2021.
- [4] Bo Wang, Qing Han, and Xiupeng Shi. Multi-Objective Optimization of ML Algorithms and FPGA Accelerators for Robotic Intelligence. In *2024 International Conference on Intelligent Robotics and Automatic Control*, pages 1–6, China, 2024. IEEE.
- [5] Thea Aarrestad, Vladimir Loncar, Nicolò Ghielmetti, Maurizio Pierini, Sioni Summers, Jennifer Ngadiuba, Christoffer Petersson, Hampus Linander, Yutaro Iiyama, Giuseppe Di Guglielmo, *et al.* Fast convolutional neural networks on FPGAs with hls4ml. *Machine Learning: Science and Technology*, 2(4):045015, 2021.
- [6] Guanhua Lu, Xingzhu Liu, Zheng Lan, and Xiupeng Shi. Co-Design of Algorithm and Chip Accelerator for Quantized Neural Network in Edge Intelligence. In *5th International Conference on Machine Learning and Intelligent Systems Engineering*, pages 1–6, China, 2025. IEEE.
- [7] Zheng Lan, Bo Wang, Senting Liu, and Xiupeng Shi. CoDANet: Hardware-Aware Neural Architecture Search for Optimized Deep Learning on FPGAs. In *2024 2nd International Conference on Artificial Intelligence and Automation Control*, pages 1–6, China, 2024. IEEE.
- [8] Bo Wang, Qing Han, and Xiupeng Shi. AutoML-Driven Co-Design of Ensemble Learning and FPGA Accelerators for Efficient Edge AI. In *2024 2nd International Conference on Artificial Intelligence and Automation Control*, pages 1–6, China, 2024. IEEE.
- [9] Xiupeng Shi, Yiik Diew Wong, Chen Chai, and Michael Zhi-Feng Li. An automated machine learning (automl) method of risk prediction for decision-making of autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 22(11):7145–7154, 2020.